Teach Module 26: Spring Configuration, Profiles, and Environments

This README captures the Module 26 teaching session, practice exercises, and lab.

Module 26 focuses on environment-based application configuration in Spring Boot.

Spring applications usually run in more than one environment:

- local development
- automated tests
- staging or QA
- production

Each environment may need different settings, such as database URLs, ports, logging levels, API keys, feature flags, cache settings, and security behavior.

The key idea is:

```
Your code should stay mostly the same across environments.
Configuration should change.
```

1. Configuration Files

Spring Boot commonly reads configuration from:

```
application.properties
application.yml
```

Example using `.properties`:

```
server.port=8080
spring.application.name=orders-service
app.tax-rate=0.0825
```

The same idea in YAML:

```
server:
  port: 8080

spring:
  application:
    name: orders-service

app:
  tax-rate: 0.0825
```

YAML is often nicer for nested configuration. `.properties` is simpler and very explicit. Both are valid.

2. Reading Configuration in Code

For a single value, you can use `@Value`:

```
@Value("${app.tax-rate}")
private BigDecimal taxRate;
```

That works, but for grouped settings, prefer `@ConfigurationProperties`.

```
@ConfigurationProperties(prefix = "app")
public class AppProperties {
    private BigDecimal taxRate;
    private String supportEmail;

    public BigDecimal getTaxRate() {
        return taxRate;
    }

    public void setTaxRate(BigDecimal taxRate) {
        this.taxRate = taxRate;
    }

    public String getSupportEmail() {
        return supportEmail;
    }

    public void setSupportEmail(String supportEmail) {
        this.supportEmail = supportEmail;
    }
}
```

Then in config:

```
app:
  tax-rate: 0.0825
  support-email: support@example.com
```

This is cleaner than scattering `@Value` fields everywhere.

3. Profiles

A Spring profile lets you activate environment-specific configuration.

Common profiles:

```
dev
test
prod
```

You can create files like:

```
application.yml
application-dev.yml
application-test.yml
application-prod.yml
```

Base config:

```yaml
spring:
  application:
    name: orders-service
```

Development config:

```yaml
# application-dev.yml
server:
  port: 8081

logging:
  level:
    com.example.orders: DEBUG
```

Production config:

```yaml
# application-prod.yml
server:
  port: 8080

logging:
  level:
    com.example.orders: INFO
```

Run with a profile:

```
java -jar app.jar --spring.profiles.active=dev
```

Or with an environment variable:

```
SPRING_PROFILES_ACTIVE=prod
```

4. Environment Variables

Environment variables are useful for settings that should not be hardcoded.

Example:

```yaml
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
```

You can also provide defaults:

```yaml
server:
  port: ${PORT:8080}
```

This means:

```
Use PORT if it exists. Otherwise use 8080.
```

5. Property Override Order

Spring can read configuration from many places. The important practical rule is:

```
More external settings usually override packaged application settings.
```

For example, this value in `application.yml`:

```yaml
server:
  port: 8080
```

Can be overridden at startup:

```
java -jar app.jar --server.port=9090
```

Or by environment variable:

```
SERVER_PORT=9090
```

This is powerful because the same `.jar` can run differently in development, test, and production.

6. Secrets Handling

Do not commit secrets into Git.

Bad:

```yaml
spring:
  datasource:
    password: my-real-production-password
```

Better:

```yaml
spring:
  datasource:
    password: ${DB_PASSWORD}
```

Secrets should usually come from:

- environment variables
- deployment platform secrets
- cloud secret managers
- Kubernetes secrets
- CI/CD secret storage

Also avoid logging secrets accidentally. Configuration is not just convenience; it is part of application security.

7. Profile-Specific Beans

Sometimes you want different Java beans depending on the environment.

Example:

```java
@Service
@Profile("dev")
public class MockEmailService implements EmailService {
    public void send(String to, String message) {
        System.out.println("Mock email sent to " + to);
    }
}
```

```java
@Service
@Profile("prod")
public class RealEmailService implements EmailService {
    public void send(String to, String message) {
        // send real email
    }
}
```

In `dev`, the app uses the mock email sender. In `prod`, it uses the real one.

8. Mental Model

Think of Spring configuration in layers:

```
Code
  lowest flexibility

application.yml
  shared defaults

application-dev.yml / application-prod.yml
  environment-specific behavior
```

```
environment variables / command-line args
  deployment-time overrides

secret manager
  sensitive runtime values
```

The goal is to make your application portable, predictable, and safe to deploy.

9. Mini Practice Example

Suppose you have this config:

```yaml
app:
  payment-provider: stripe
  retry-count: 3
```

And this production override:

```yaml
app:
  retry-count: 5
```

When the `prod` profile is active, Spring uses:

```
payment-provider = stripe
retry-count = 5
```

The production file overrides only the value it defines.

10. Key Takeaway

Module 26 is about separating application behavior from environment-specific settings.

A strong Spring developer knows how to:

- keep config outside code
- use profiles cleanly
- override settings safely
- avoid committing secrets
- make one app artifact run across multiple environments

Practice Exercises

Exercise 1: Basic App Configuration

Create a Spring Boot app with these values in `application.yml`:

```yaml
app:
  name: Order Service
```

```
  version: 1.0
  support-email: support@example.com
```

Create a REST endpoint:

```
GET /config
```

It should return those values as JSON.

Practice goal: reading custom configuration from Spring Boot.

Exercise 2: Use `@ConfigurationProperties`

Create a class called `AppProperties` that maps this config:

```
app:
  name: Order Service
  retry-count: 3
  feature-flags:
    payments-enabled: true
    coupons-enabled: false
```

Then expose it through:

```
GET /app-settings
```

Practice goal: grouping related config cleanly instead of using many `@Value` fields.

Exercise 3: Create Dev and Prod Profiles

Create:

```
application-dev.yml
application-prod.yml
```

In `dev`:

```
app:
  environment-name: Development
  debug-mode: true
```

In `prod`:

```
app:
  environment-name: Production
  debug-mode: false
```

Run the app twice:

```
java -jar app.jar --spring.profiles.active=dev
```

```
java -jar app.jar --spring.profiles.active=prod
```

Practice goal: seeing how profiles change application behavior.

Exercise 4: Profile-Specific Beans

Create an interface:

```
public interface NotificationService {
    String send(String message);
}
```

Create two implementations:

```
@Profile("dev")
@Service
public class ConsoleNotificationService implements NotificationService {
    public String send(String message) {
        return "DEV notification: " + message;
    }
}
```

```
@Profile("prod")
@Service
public class EmailNotificationService implements NotificationService {
    public String send(String message) {
        return "PROD email sent: " + message;
    }
}
```

Expose:

```
POST /notify
```

Practice goal: using different Spring beans in different environments.

Exercise 5: Environment Variable Override

Add this config:

```
app:
  external-api-url: ${EXTERNAL_API_URL:http://localhost:9999}
```

Run once without the environment variable. Then run again with:

```
EXTERNAL_API_URL=https://api.example.com
```

Practice goal: using environment variables with fallback defaults.

Exercise 6: Configure Database Per Profile

Use H2 for `dev` and PostgreSQL-style config for `prod`.

`application-dev.yml`:

```yaml
spring:
  datasource:
    url: jdbc:h2:mem:testdb
    username: sa
    password:
```

`application-prod.yml`:

```yaml
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
```

Practice goal: understanding why local development and production should not share database settings.

Exercise 7: Logging Per Environment

Set logging differently by profile.

`application-dev.yml`:

```yaml
logging:
  level:
    com.example: DEBUG
```

`application-prod.yml`:

```yaml
logging:
  level:
    com.example: INFO
```

Add a controller that logs debug and info messages.

Practice goal: controlling log verbosity by environment.

Exercise 8: Secrets Safety Check

Create a fake bad config file with hardcoded credentials, then refactor it to use environment variables.

Bad:

```yaml
payment:
  api-key: sk_live_123456
```

Better:

```yaml
payment:
  api-key: ${PAYMENT_API_KEY}
```

Practice goal: learning what should never be committed to source control.

Exercise 9: Command-Line Override

Set this in `application.yml`:

```yaml
server:
  port: 8080
```

Then run:

```
java -jar app.jar --server.port=9090
```

Practice goal: understanding Spring Boot override priority.

Exercise 10: Mini Project

Build a small Product Catalog API with:

```
GET /products
GET /config
POST /notifications
```

Requirements:

- `dev` profile uses in-memory sample products
- `prod` profile reads database settings from environment variables
- app name and support email come from config
- notification behavior changes by profile
- no secrets are hardcoded
- logging level changes by profile

This is the best end-to-end practice because it combines configuration files, profiles, environment variables, beans, and safe secret handling.

Lab: Spring Configuration, Profiles, and Environments

Lab Goal

Build a small Spring Boot application that changes behavior based on environment configuration.

You will practice:

- `application.yml`
- `application-dev.yml`
- `application-prod.yml`
- `@ConfigurationProperties`
- Spring profiles
- environment variables
- profile-specific beans
- safe secret handling

Lab Scenario

You are building a small Order Service. The app should behave differently in development and production.

In `dev`, it should use mock behavior.

In `prod`, it should expect real external configuration from environment variables.

Step 1: Create Configuration Files

Create or update:

```
src/main/resources/application.yml
src/main/resources/application-dev.yml
src/main/resources/application-prod.yml
```

In `application.yml`:

```yaml
spring:
  application:
    name: order-service

app:
  support-email: support@example.com
  retry-count: 3
  external-api-url: ${EXTERNAL_API_URL:http://localhost:9999}
```

In `application-dev.yml`:

```yaml
app:
  environment-name: Development
  debug-mode: true

logging:
  level:
    com.example: DEBUG
```

In `application-prod.yml`:

```yaml
app:
  environment-name: Production
  debug-mode: false
```

```
  external-api-url: ${EXTERNAL_API_URL}

logging:
  level:
    com.example: INFO
```

Step 2: Create `AppProperties`

Create:

```
@ConfigurationProperties(prefix = "app")
@Component
public class AppProperties {
    private String supportEmail;
    private int retryCount;
    private String externalApiUrl;
    private String environmentName;
    private boolean debugMode;

    // getters and setters
}
```

Step 3: Create a Config Controller

Create an endpoint:

```
GET /config
```

It should return:

```
{
  "supportEmail": "support@example.com",
  "retryCount": 3,
  "externalApiUrl": "http://localhost:9999",
  "environmentName": "Development",
  "debugMode": true
}
```

Use `AppProperties` in the controller.

Step 4: Create Profile-Specific Notification Services

Create an interface:

```
public interface NotificationService {
    String send(String message);
}
```

Create a dev implementation:

```java
@Service
@Profile("dev")
public class ConsoleNotificationService implements NotificationService {
    public String send(String message) {
        return "DEV notification logged: " + message;
    }
}
```

Create a prod implementation:

```java
@Service
@Profile("prod")
public class EmailNotificationService implements NotificationService {
    public String send(String message) {
        return "PROD email notification sent: " + message;
    }
}
```

Step 5: Create Notification Endpoint

Create:

```
POST /notify
```

Example request:

```
{
  "message": "Order #123 has shipped"
}
```

Example dev response:

```
DEV notification logged: Order #123 has shipped
```

Example prod response:

```
PROD email notification sent: Order #123 has shipped
```

Step 6: Run with Dev Profile

Run:

```
mvn spring-boot:run -Dspring-boot.run.profiles=dev
```

Then test:

```
GET http://localhost:8080/config
POST http://localhost:8080/notify
```

Expected result:

- environment is `Development`
- debug mode is `true`
- notification service uses dev behavior

Step 7: Run with Prod Profile

Set an environment variable first.

PowerShell:

```
$env:EXTERNAL_API_URL="https://api.company.com"
```

Then run:

```
mvn spring-boot:run -Dspring-boot.run.profiles=prod
```

Expected result:

- environment is `Production`
- debug mode is `false`
- external API URL comes from environment variable
- notification service uses prod behavior

Step 8: Validation Checklist

Your lab is complete when:

- `/config` returns values from configuration
- `dev` and `prod` profiles return different values
- notification behavior changes by profile
- `EXTERNAL_API_URL` can override config
- no secret or production value is hardcoded
- app runs successfully with both profiles

Bonus Challenge

Add this config:

```
payment:
  api-key: ${PAYMENT_API_KEY}
```

Then create:

```
GET /payment/status
```

It should return:

```
Payment provider configured
```

But it must never return or print the actual API key.